PROGRAMAÇÃO PARALELA E SISTEMAS EM TEMPO REAL



sagah

Divisão e conquista em programação paralela

Juliane Soares

OBJETIVOS DE APRENDIZAGEM

- Conceituar divisão e conquista em programação paralela.
- Descrever métodos de divisão e conquista.
- Identificar casos de divisão e conquistas.

Introdução

O uso da tecnologia está presente em nosso dia a dia, seja na área pessoal, seja na profissional. Com o aumento do uso da tecnologia, são necessários dispositivos e sistemas cada vez mais potentes, a fim de atender à demanda exigida. Assim, problemas complexos exigem soluções complexas.

A programação paralela permite que um programa seja executado rapidamente. Ela ajuda os desenvolvedores a dividirem tarefas em menores segmentos de trabalho para que um programa consiga executá-las simultaneamente, concluindo o seu processamento com maior eficiência. Apesar de exigir um esforço inicial maior por parte dos desenvolvedores na criação de algoritmos e códigos paralelos eficientes, em geral se economiza tempo, pois se aproveita o poder de processamento paralelo, executando o programa em vários nós de computação e núcleos de processadores ao mesmo tempo.

Neste capítulo, você vai conhecer uma das principais abordagens de programação paralela: divisão e conquista. Além disso, vai estudar os métodos existentes para essa técnica. Por fim, vai ver casos práticos de divisão e conquista.

Divisão e conquista: conceitos

O termo programação paralela pode ser utilizado em conjunto com o termo computação paralela, pois se refere aos sistemas de alta eficiência. A programação paralela é o processo que decompõe um problema em tarefas menores, de modo que sejam executadas simultaneamente, utilizando vários recursos de computação. Ou seja, com o uso da programação paralela as tarefas podem ser paralelizadas, sendo executadas ao mesmo tempo a partir do uso de mais computadores ou de vários núcleos de um processador. Esse tipo de programação é importante para grandes projetos, em que a velocidade e a precisão são necessárias. Apesar de ser uma tarefa complexa, ela é fundamental, pois permite que desenvolvedores, pesquisadores e usuários consigam realizar pesquisas e análises de forma mais rápida e eficiente em comparação com os programas capazes de processar somente uma tarefa por vez.

Existem alguns modelos de aplicação da programação paralela, e o *divid--and-conquer* (divisão e conquista) é um dos mais comuns (JAVATPOINT, c2022). Hardwick (1997) afirma que o método de divisão e conquista é a técnica mais conhecida de projetos de algoritmos, pois, por meio dela, é possível resolver de forma mais simples um problema maior. Isso porque essa técnica se refere à resolução de um grande problema dividindo-o em um ou mais subproblemas menores, de modo que as resoluções de cada um desses subproblemas sejam unidas para gerar a solução do problema original. Esses algoritmos funcionam em três etapas. A princípio, o problema é dividido em subproblemas menores. Em seguida, vem a etapa da conquista, em que os subproblemas são resolvidos de modo recursivo até que sejam suficientemente pequenos para serem resolvidos diretamente. Por fim, a última etapa é a combinação, em que a solução dos subproblemas são combinadas para se chegar à solução do problema original.

A Figura 1 representa um problema dividido em dois subproblemas. Note que a solução de cada um deles se une, gerando a solução do problema.

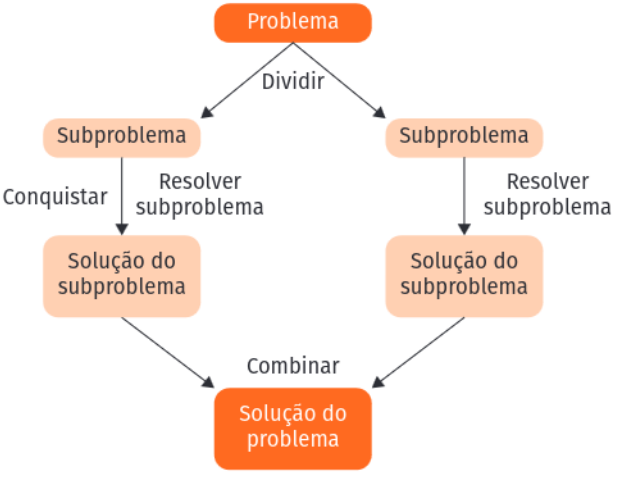


Figura 1. Passos do método de divisão e conquista.
Fonte: Adaptada de Cormen e Balkcom (c2022).

A Figura 2 mostra um exemplo de um algoritmo de divisão e conquista fazendo múltiplas chamadas recursivas. Um problema é dividido em dois subproblemas, e estes são divididos em dois subproblemas cada um, gerando quatro subproblemas. Esses quatro subproblemas são divididos em mais dois subproblemas cada um, fechando em oito subproblemas, o que resulta em oito resoluções. Tais resoluções são combinadas de duas em duas, repetindo o processo até que seja encontrada uma solução única, sendo esta a solução do problema original.

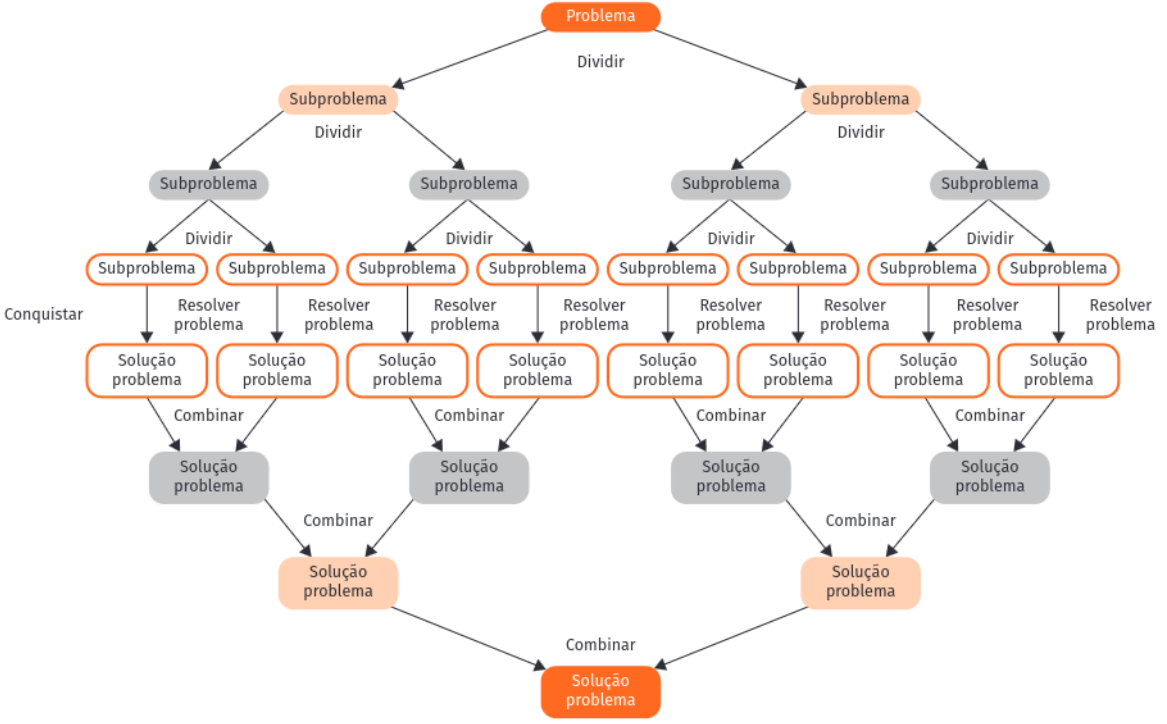


Figura 2. Algoritmo de divisão e conquista com múltiplas chamadas recursivas.
Fonte: Adaptada de Cormen e Balkcom (c2022).

O método de divisão e conquista pode ser considerado uma variante da técnica de programação de cima para baixo mais geral. No entanto, a diferença está no fato de que os subproblemas são instâncias do mesmo problema, podendo ser expressos recursivamente. Isso significa que o mesmo algoritmo pode ser aplicado aos subproblemas do problema original. Contudo, como o algoritmo recursivo é encerrado?

Hardwick (1997) afirma que é preciso um caso base para encerrar a recursão. Isso pode ser feito testando se o problema é pequeno o suficiente para ser resolvido por método direto, como no *quicksort* (algoritmo de ordenação por divisão e conquista), em que o caso base é alcançado quando há 0 ou 1 elementos na lista. Quando se chega a esse ponto é porque a lista está ordenada. Assim, para que o problema seja resolvido, se retorna para a lista de entrada.

Além disso, também é necessária uma fase de divisão, para que o problema seja dividido em subproblemas, e uma fase de combinação, para que as soluções dos subproblemas sejam combinadas em uma solução do problema original.



Exemplo

No *Quicksort*, a fase da divisão divide a lista em três listas, onde estão contidos os elementos menores, iguais e maiores que o pivô. Por fim, a fase de combinação anexa as subsoluções classificadas em cada um dos lados da lista que contém elementos iguais.

De acordo com Hardwick (1997), a estrutura de caso base, o solucionador direto e as fases de divisão e combinação podem ser generalizados em um modelo básico para algoritmos de divisão e conquista, conforme mostra o pseudocódigo a seguir.

```
function d.and.c
 if basecase
 then
  return solve
 else

(p1,...,pn ) = divide(p)
  return combine(d.and.c(, ..., d.and.c
 end if
```

Esse pseudocódigo é utilizado como referência para vários outros métodos que podem diferenciar algoritmos de divisão e conquista para sua implementação em arquiteturas paralelas. Vamos conhecer esses métodos na próxima seção.

Divisão e conquista: métodos

Nesta seção, vamos estudar os métodos de divisão e conquista em programação paralela de acordo com Hardwick (1997).

Fator de ramificação

O fator de ramificação de um algoritmo de divisão e conquista refere-se ao número de subproblemas em que um problema é dividido. Esse pode ser considerado o grau de recursão, e é uma forma de classificar esses algoritmos.

Para ser um verdadeiro algoritmo de divisão e conquista e o problema ser dividido de fato, o fator de ramificação deve ser de dois ou mais. Fatores de ramificação mais altos trazem uma complexidade adicional aos algoritmos de divisão e conquista. No entanto, o desempenho necessário para compensar essa complexidade pode trazer vantagens, já que um fator de ramificação mais elevado pode significar a necessidade de menos etapas recursivas, tendo um caso base mais eficiente.

Equilíbrio

Para um algoritmo de divisão e conquista ser considerado balanceado, ele deve dividir o problema inicial em subproblemas que tenham o mesmo tamanho, em casos em que os subproblemas somem o tamanho do problema original.

Algoritmos balanceados apresentam a vantagem de não necessitar de balanceamento de carga. Isso significa que, com o número de processadores sendo uma potência do fator de ramificação do algoritmo e a quantidade do trabalho realizado dependendo da quantidade de dados, os processadores envolvidos no processo terão iguais quantidades de trabalho.

No entanto, para que seja implementado um algoritmo de divisão e conquista não balanceado, é necessário adicionar alguma forma de balanceamento de carga, gerando maior complexidade aos algoritmos. Por isso, são poucos os modelos de programação que têm a capacidade de executar algoritmos de divisão e conquista não balanceados.

Divisibilidade embaraçosa

Para ter divisibilidade embaraçosa, a etapa de divisão de um algoritmo de divisão e conquista deve ser executada em tempo constante. Essa forma é extremamente restritiva, pois quase nenhuma comunicação é necessária entre os processadores.

Em algoritmos de divisibilidade embaraçosa, o problema pode ser dividido em dois ou mais subproblemas de forma imediata, não exigindo movimento extra de dados na etapa de divisão.

Dependência de dados da função de divisão

Para um algoritmo de divisão e conquista ter uma função de divisão dependente de dados, os tamanhos relativos dos subproblemas devem depender de alguma forma dos dados de entrada. Essa subclasse representa a maior parte dos algoritmos não balanceados. No entanto, algoritmos não balanceados também podem não ter dependência de dados da função de divisão caso o tamanho de sua estrutura tenha relação previsível com os tamanhos dos subproblemas, desde que não sejam iguais.

Dependência de dados da função de tamanho

Quando o tamanho total dos subproblemas de um algoritmo de divisão e conquista tem algum nível de dependência dos dados de entrada, ele vai ter uma função de tamanho dependente dos dados. Esses algoritmos formam uma subclasse. Tal modelo de algoritmo adiciona ou descarta elementos com base nos dados de entrada, para tentar reduzir o tamanho do problema.

Um exemplo que pode ser usado para explicar melhor essa dependência são os algoritmos de classificação rápida de três vias, em que os dados são divididos em elementos menores, iguais e maiores que o pivô. Há uma função de tamanho dependente de dados, pois ele apenas se repete em elementos menores e maiores que o pivô.

Algoritmos de divisão e conquista com dependência de dados da função de tamanho podem ser mais complexos para implementação, pois os desequilíbrios de carga podem surgir na quantidade total dos dados em um processador, e não apenas em seu particionamento.

Paralelismo ou sequencialidade de controle

O paralelismo ou sequencialidade de controle em um algoritmo de divisão e conquista ocorre quando os subproblemas devem seguir uma determinada ordem de execução. Essa ordenação ocorre quando uma subtarefa depende do resultado da anterior para ser calculada.

No entanto, essa ordenação de subproblema elimina qualquer possibilidade de paralelismo de controle a partir da execução de duas ou mais subtarefas simultaneamente, sendo necessária a adição de paralelismo de dados para alcançar a aceleração das tarefas.

Paralelismo de dados

Para que um algoritmo de divisão e conquista seja paralelo aos dados, as operações de divisão, conquista e combinação não podem ter gargalos seriais. É importante salientar que, caso o paralelismo não esteja presente nessas etapas, o algoritmo será lento, não entregando a aceleração necessária para a melhoria de desempenho dos programas.

O paralelismo de dados geralmente está presente no estágio de divisão, que é baseado em estrutura ou valor, podendo ser implementado paralelamente aos dados. Apesar de a fase de divisão ser de dados paralelos e as subtarefas recursivas não serem ordenadas (podendo ser executadas paralelamente), o estágio final de mesclagem deve ser serializado para a comparação de elementos sequenciais das duas subsequências ordenadas, eliminando os gargalos do algoritmo.

Divisão e conquista: casos

Em uma abordagem de divisão e conquista, o processamento é realizado dividindo sucessivamente a tarefa a ser executada. A princípio, a raiz avalia se o trabalho pode ser desempenhado localmente. Caso contrário, realiza as subdivisões dos trabalhos (PASINI, 2007).

A técnica de divisão e conquista apresenta algumas vantagens, conforme aponta JavaTpoint (c2022).

- Esse algoritmo divide o problema principal em subproblemas, de modo que seja possível resolvê-lo de forma muito mais rápida em comparação a outros tipos de algoritmos.

- Faz uso da memória cache eficientemente, sem que ocupe muito espaço, já que são resolvidos subproblemas simples dentro da memória cache no lugar de acessar a memória principal, o que tornaria o processo mais lento.
- Algoritmos são tratados por sistemas que incorporam processamento paralelo.

Apesar das vantagens, essa técnica também apresenta desvantagens, de acordo com JavaTpoint (c2022).

- A maioria de seus algoritmos é projetada com recursão incorporada, então precisa de um alto gerenciamento de memória.
- Uma pilha explícita pode utilizar muito espaço.
- Se a recursão executada for maior que a pilha presente na CPU, ela pode travar o sistema.

JavaTpoint (c2022) também apresenta os principais algoritmos que têm como base a abordagem de divisão e conquista; veja a seguir.

- Pesquisa binária: é um algoritmo de pesquisa que funciona comparando o valor de destino com o elemento do meio que existe em uma matriz classificada. O processo é repetido até que o valor alvo seja alcançado. Nesse caso, o algoritmo de divisão e conquista serve para dividir o espaço da busca pela metade a cada iteração.
- *Quicksort*: é um algoritmo de ordenação eficiente. Ele inicia selecionando um valor pivô de uma matriz e depois divide os demais elementos da matriz em duas submatrizes. A partição é realizada por meio da comparação entre cada um dos elementos com o valor do pivô, comparando se o elemento contém um valor maior ou menor que o pivô e, em seguida, classificando os *arrays* recursivamente.
- *Mergesort*: algoritmo de ordenação por meio da comparação de *arrays*. Divide uma matriz em submatriz e, em seguida, classifica de modo recursivo cada uma delas. Depois dessa classificação, o algoritmo as mescla novamente.
- Algoritmo de Strassen: algoritmo de multiplicação de matrizes muito rápido quando trabalha em grandes matrizes.
- Algoritmo Cooley-Tukey Fast Fourier Transform (FFT): algoritmo que segue a abordagem de divisão e conquista, impondo complexidade de 0(N log N).

- Algoritmo Karatsuba para multiplicação rápida: multiplica dois números de *n* dígitos, reduzindo-os ao máximo para alcançar um único dígito.

A Figura 3 representa o funcionamento de uma torre de Hanói, um exemplo que utiliza a técnica de divisão e conquista para sua resolução.

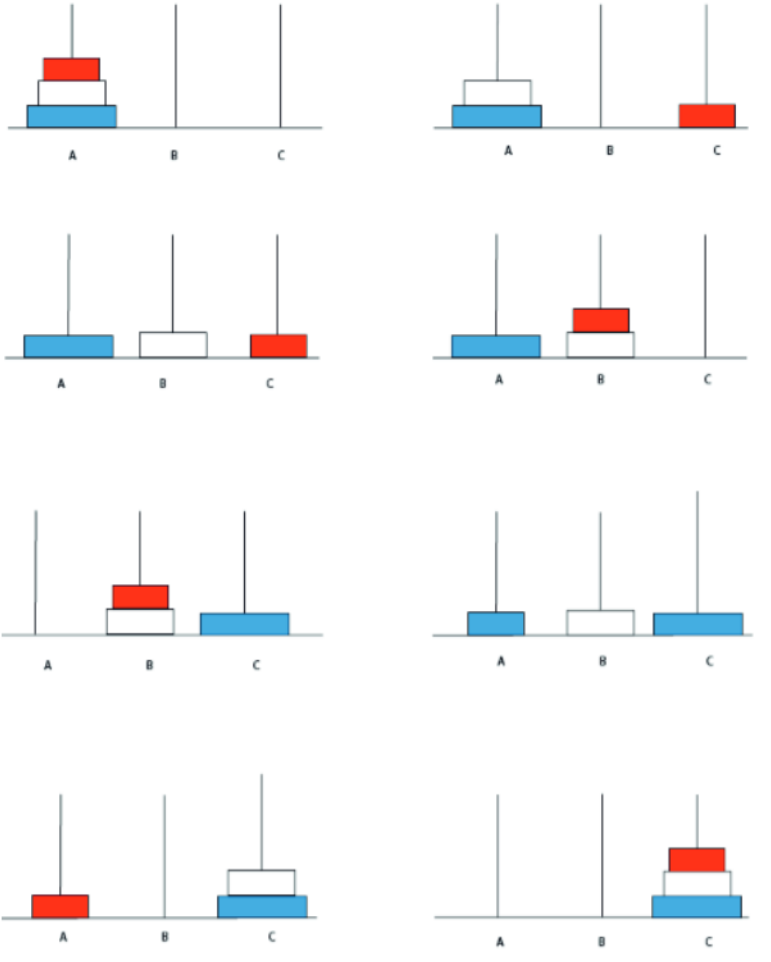


Figura 3. Torre de Hanói.
Fonte: JavaTpoint (c2022, documento *on-line*).

Nas torres de Hanói, uma pilha de discos de tamanhos diferentes é movida de um local para outro. Existem algumas restrições: somente um disco por vez pode ser movido, e um disco maior nunca pode ser colocado sobre um menor. Uma solução de divisão e conquista resolve o problema maior, resolvendo, primeiramente, uma versão menor do mesmo tipo de problema. Para mover uma pilha de *n* discos para o local C, primeiro se move a pilha menor de *n*-1 discos para o local B. Deve-se seguir o mesmo tipo de procedimento para os *n* discos e para os *n*-1 discos. Essa abordagem é recursiva, pois, para a pilha grande ser movida, o mesmo procedimento deve primeiro ser usado para mover a pilha menor sobre o disco maior.

Assim como as outras abordagens existentes na programação paralela, a técnica de divisão e conquista serve para aumentar o desempenho de computadores e sistemas. Dividir grandes problemas em subproblemas para sua resolução mais rápida pode ser uma tarefa complexa, mas seu resultado traz inúmeros benefícios, principalmente considerando o mundo imediatista em que vivemos.

Referências

CORMEN, T.; BALKCOM, D. Algoritmos de divisão e conquista. *Khan Academy*, c2022. Disponível em: https://pt.khanacademy.org/computing/computer-science/algorithms/merge-sort/a/divide-and-conquer-algorithms. Acesso em: 13 jun. 2022.

HARDWICK, J. C. *Practical parallel divide-and-conquer algorithms*. 1997. Doctoral Thesis (Computer Science) – School of Computer Science, Pittsburgh, 1997.

JAVATPOINT. Divide and conquer. *JavaTpoint*, c2022. Disponível em: https://www.javatpoint.com/divide-and-conquer-introduction. Acesso em: 13 jun. 2022.

PASINI, F. *Desenvolvimento de aplicações paralelas a partir de modelos em gramática de grafos baseada em objetos*. 2007. Dissertação (Mestrado em Ciência da Computação) – Pontifícia Universidade Católica do Rio Grande do Sul, Porto Alegre, 2007.

Fique atento

Os *links* para *sites* da *web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

Conteúdo:

sagah